

PROYECTO II

Simulador de Sistema Operativo (Mini-Windows)

1. Objetivo general

El proyecto consiste en desarrollar una aplicación de escritorio que simula el comportamiento de un sistema operativo (llamado "Mini-Windows"), replicando funciones típicas como la gestión de archivos, un editor de texto, un visor de imágenes, una consola de comandos y un reproductor musical.

Dentro de este mismo sistema se debe integrar un módulo adicional llamado INSTA+, una red social inspirada en Instagram que reutiliza la misma lógica de almacenamiento en archivos binarios usada por el resto del sistema.

Para que ambos módulos funcionen de forma correcta, ordenada y eficiente, el proyecto exige aplicar cuatro conceptos técnicos centrales que se explican en la siguiente sección antes de entrar en el detalle funcional de cada módulo:

- Archivos binarios, para el almacenamiento permanente de la información.
- Hilos (threads), para ejecutar tareas simultáneas sin congelar la interfaz.
- Sockets cliente-servidor, para permitir que el sistema se comuniquen en red.
- Listas enlazadas simples, para organizar y recorrer datos en memoria.

2. Conceptos técnicos fundamentales del proyecto

Antes de describir cada módulo, es importante entender con claridad los cuatro pilares técnicos sobre los que se construye todo el proyecto. Cada uno se explica con su definición, su propósito general y la forma concreta en que debe aplicarse dentro de Mini-Windows e INSTA+.

2.1 Archivos binarios

Un archivo binario almacena la información como una secuencia de bytes en el formato exacto en que la maneja el programa (números, texto, objetos), en lugar de guardarla como texto legible. Esto lo diferencia de un archivo de texto plano (.txt), que solo contiene caracteres legibles por una persona.

¿Por qué se usan en este proyecto?

- **Persistencia de datos:** toda la información del sistema (usuarios, seguidores, publicaciones, mensajes) debe sobrevivir al cierre del programa, por lo que se guarda en disco en archivos binarios en vez de mantenerse solo en memoria.
- **Registros estructurados:** cada archivo guarda "registros" con una estructura fija (por ejemplo, un usuario siempre tiene nombre, username, password, edad, etc.), lo cual es ideal para el formato binario.

Programación II

- **Eficiencia:** leer y escribir en binario es más rápido y ocupa menos espacio que convertir todo constantemente a texto.

Cómo aplicarlo en el proyecto

- Definir una clase por cada tipo de registro (Usuario, Publicacion, Mensaje, Sticker) que implemente Serializable.
- Usar ObjectOutputStream / ObjectInputStream para escribir y leer objetos completos, o RandomAccessFile cuando se necesite acceso directo a un registro específico dentro del archivo.
- Cada archivo del proyecto (users.ins, followers.ins, following.ins, insta.ins, inbox.ins, stickers.ins, usuarios.sop) debe manejarse siempre como archivo binario, nunca como texto plano.

2.2 Hilos (Threads)

Un hilo es una unidad de ejecución dentro de un programa. Un programa puede tener varios hilos ejecutándose "al mismo tiempo" (de forma concurrente), lo cual permite que la aplicación siga respondiendo mientras realiza tareas que tardan, en lugar de quedarse congelada.

¿Dónde se necesitan hilos en este proyecto?

- **Reproductor musical:** la reproducción de un archivo .mp3 debe correr en su propio hilo, para que el usuario pueda seguir usando el resto del sistema (navegar carpetas, escribir en el editor) mientras la música suena.
- **Organizador de archivos:** la función "organizar", que recorre y clasifica archivos por tipo, puede tardar en carpetas grandes; ejecutarla en un hilo aparte evita que la interfaz se bloquee.
- **Notificaciones del Inbox:** revisar si llegaron mensajes o stickers nuevos debe hacerse en segundo plano, con un hilo que consulte periódicamente el archivo inbox.ins.
- **Carga de imágenes:** cargar o mostrar colecciones de imágenes (visor de imágenes, feed de INSTA+) se beneficia de un hilo dedicado para no congelar la ventana principal.

Cuidados importantes

- Cuando dos hilos puedan escribir sobre el mismo archivo binario al mismo tiempo (por ejemplo, dos procesos que actualizan followers.ins), el acceso debe protegerse con bloques synchronized o un mecanismo de bloqueo (Lock), para evitar que la información se corrompa.
- Cada hilo debe tener una responsabilidad clara y comunicarse con la interfaz principal de forma segura (por ejemplo, mediante invokeLater en aplicaciones Swing).

2.3 Sockets: arquitectura cliente-servidor

Un socket es el punto de conexión que permite que dos programas se comuniquen a través de una red, usando el protocolo TCP/IP. En una arquitectura cliente-servidor, un programa servidor queda escuchando conexiones en un puerto determinado, y uno o varios programas cliente se conectan a él para enviar y recibir información.

Aplicación en el proyecto

Para que Mini-Windows e INSTA+ puedan funcionar de forma multiusuario en red (por ejemplo, que varias personas usen la red social desde distintas computadoras conectadas a un mismo servidor central), se propone la siguiente arquitectura:

Programación II

- **Servidor:** un programa que usa ServerSocket para quedar escuchando en un puerto fijo. Por cada cliente que se conecta, el servidor crea un nuevo hilo (ver sección 2.2) dedicado a atender esa conexión sin bloquear a los demás usuarios.
- **Cliente:** cada instancia de la aplicación INSTA+ usa la clase Socket para conectarse a la dirección IP y puerto del servidor, y desde ahí envía solicitudes como iniciar sesión, publicar un insta, seguir a alguien o enviar un mensaje al Inbox.
- **Protocolo de comunicación:** se recomienda definir comandos de texto simples que viajen entre cliente y servidor, por ejemplo LOGIN;usuario;password, POST;contenido, FOLLOW;username, de modo que el servidor los interprete y responda con el resultado de la operación.
- **Acceso a los archivos binarios:** el servidor es quien realmente lee y escribe los archivos .ins; los clientes nunca acceden a los archivos directamente, sino que le piden al servidor que lo haga por ellos, evitando conflictos de acceso simultáneo.

2.4 Listas enlazadas simples

Una lista enlazada simple es una estructura de datos formada por nodos; cada nodo guarda un dato y una referencia (o "enlace") al siguiente nodo de la lista. A diferencia de un arreglo, no necesita un tamaño fijo desde el inicio y permite insertar o eliminar elementos con facilidad.

Dónde se aplica en el proyecto

- **Timeline de publicaciones:** los registros leídos desde insta.ins pueden cargarse en una lista enlazada donde cada nodo es un insta (publicación), permitiendo recorrerla del más reciente al más antiguo sin necesidad de un arreglo de tamaño fijo.
- **Listado de followers/following:** los usernames leídos de followers.ins y following.ins se pueden representar como una lista enlazada, lo cual facilita agregar un nuevo seguidor o eliminar un "dejar de seguir" sin reorganizar todo un arreglo.
- **Resultados de búsqueda:** tanto la búsqueda de personas como la búsqueda por hashtag arman su resultado recorriendo y agregando coincidencias a una lista enlazada.
- **Organizador de archivos:** al clasificar archivos por tipo (imágenes, documentos, música) dentro del navegador de Mini-Windows, cada categoría puede mantenerse como una lista enlazada de rutas de archivo.

El documento original sugiere apoyarse en un ArrayList de Java para extraer y ordenar los datos antes de mostrarlos; sin embargo, como parte de los requisitos académicos del curso, se pide implementar y usar explícitamente una lista enlazada simple propia (con clases Nodo y ListaEnlazada) en al menos los puntos anteriores, en lugar de depender únicamente de las colecciones ya incluidas en Java.

3. Módulo 1 — Mini-Windows (simulador de sistema operativo)

Este módulo simula las funciones básicas de un sistema operativo de escritorio, incluyendo la gestión de usuarios, archivos, texto, imágenes, comandos y música.

3.1 Configuración inicial y estructura raíz

Programación II

- El sistema cuenta por defecto con una unidad Z:\, que funciona como directorio raíz de todo el sistema de archivos simulado.
- Al iniciar el sistema por primera vez debe existir, por defecto, un usuario administrador ya creado, listo para usarse.

3.2 Soporte multiusuario

El sistema debe permitir múltiples usuarios, cada uno con su propio espacio de archivos aislado:

- Al crear un nuevo usuario del sistema, se debe generar automáticamente una subcarpeta "Z:\usuario", es decir, un subdirectorío raíz exclusivo para cada usuario.
- Cada usuario solo puede iniciar sesión e ingresar a su propio sistema de archivos, usando su usuario y contraseña.
- **Usuario administrador:** puede crear nuevos usuarios y acceder a las carpetas de todos los usuarios existentes del sistema.
- **Usuario estándar:** solo puede ver y modificar el contenido de su propia cuenta.
- Al crearse, cada usuario recibe automáticamente 3 carpetas básicas: Mis Documentos, Música y Mis Imágenes.

El sistema debe mostrar una barra administradora principal desde la cual se acceda a todas las herramientas (explorador de archivos, editor de texto, visor de imágenes, consola, reproductor).

3.3 Navegador y organizador de archivos

El explorador de archivos debe presentarse mediante un componente tipo árbol (JTree), mostrando las carpetas y archivos existentes a partir de la carpeta raíz del usuario.

- **Función "Organizar":** al seleccionar una carpeta y ejecutar esta función, el sistema debe clasificar automáticamente los archivos de imágenes, documentos y música, colocando cada tipo en su propia subcarpeta.
- **Ordenamiento:** el sistema debe poder ordenar los archivos por nombre, fecha, tipo o tamaño.
- **Gestión de archivos:** debe ser posible renombrar, crear, copiar y pegar archivos o carpetas.

3.4 Editor de texto con formato

Se debe incluir un editor capaz de crear y modificar archivos .txt, con una barra de herramientas que permita aplicar formato al texto. Este formato debe conservarse y visualizarse cada vez que el archivo se vuelva a abrir.

Como mínimo, el editor debe permitir cambiar:

- Color del texto
- Tipo de fuente
- Tamaño de fuente

3.5 Visor de imágenes

El sistema debe incluir un visor que permita recorrer una colección de imágenes dentro de una carpeta, con botones de "Siguiente" y "Anterior", de forma similar al visor de imágenes clásico de Windows.

3.6 Consola de comandos

Debe implementarse una consola que imite el funcionamiento del CMD de Windows, mostrando siempre la carpeta actual y soportando los siguientes comandos:

Comando	Descripción
mkdir <nombre>	Crea una nueva carpeta
rm <nombre>	Elimina una carpeta
cd <carpeta>	Cambia a la carpeta indicada
cd..	Regresa a la carpeta anterior
dir	Lista todas las carpetas y archivos de la carpeta actual
date	Muestra la fecha actual
time	Muestra la hora actual

3.7 Reproductor de música

El reproductor debe permitir reproducir archivos .mp3 obtenidos desde el navegador de archivos, con las funciones mínimas:

- Play
- Pause
- Stop
- Lista de las canciones.
- Caratula de la canción
- Descripción.

4. Módulo 2 — Red social INSTA+

INSTA+ es una red social integrada dentro de Mini-Windows, inspirada en Instagram, que reutiliza el manejo de archivos binarios del sistema. A continuación se describe cada parte de forma detallada, integrando tanto la especificación original como sus mejoras posteriores (Inbox, Stickers y diseño responsive).

4.1 Visión general

- Toda la aplicación debe funcionar en una sola pantalla principal, replicando la experiencia de uso de la red social real (sin ventanas emergentes separadas para cada función).

Programación II

- Los datos se organizan en archivos binarios independientes por usuario, dentro de una estructura de carpetas ordenada.
- El almacenamiento debe usar exclusivamente archivos binarios (nunca texto plano) para nombres, contraseñas, publicaciones y mensajes.

4.2 Pantalla de inicio: Login y creación de cuenta

La pantalla inicial le presenta al usuario dos opciones: Log In o Crear cuenta.

a) Log In

- El usuario ingresa su username y password.
- Si la autenticación es correcta, se muestra el menú principal del sistema.
- Si los datos son incorrectos, se muestra un mensaje de error y se pregunta si desea repetir el login o ir a crear una cuenta nueva.

b) Crear cuenta

Para crear una cuenta se solicitan los siguientes datos, que alimentan el archivo binario users.ins (el registro maestro con la información de todos los usuarios del sistema):

Campo	Tipo de dato	Detalle
Nombre completo	STRING	Nombre del usuario
Género	CHAR	'M' o 'F'
Username	STRING	Debe ser único en todo el sistema
Password	STRING	Contraseña de acceso
Fecha de registro	DATE	Se toma automáticamente del sistema
Edad	INT	Edad del usuario
Estado de la cuenta	BOOLEAN	Activa por defecto
Foto de perfil	Archivo/Ruta	Imagen que se muestra en el perfil

4.3 Estructura de carpetas y archivos binarios

Al registrarse, se crea automáticamente una carpeta con el nombre exacto del username dentro de una carpeta raíz general de la red social (INSTA_RAIZ). Esa carpeta personal contiene los archivos binarios propios del usuario:

Archivo	Contenido
following.ins	Usernames de las cuentas que el usuario sigue
followers.ins	Usernames de las cuentas que siguen al usuario (sin duplicados)
insta.ins	Publicaciones ("insta") del usuario: autor, fecha y contenido (máx. 140 caracteres)
inbox.ins	Mensajes privados enviados y recibidos

Archivo	Contenido
stickers.ins	Registro de los stickers disponibles para ese usuario

Además de la carpeta personal, cada usuario cuenta con las siguientes subcarpetas:

- /imagenes — publicaciones e imágenes subidas por el usuario.
- /folders_personales — carpetas creadas libremente por el usuario (por ejemplo, Viajes, Familia, Memes) para organizar sus imágenes.
- /stickers_personales — stickers propios importados por el usuario.

A nivel general del sistema (fuera de las carpetas de cada usuario) existen además:

- users.ins — registro binario con todos los usuarios del sistema.
- stickers_globales — carpeta con los stickers por defecto disponibles para todos los usuarios.

4.4 Pantalla principal (menú único)

Una vez autenticado, el usuario ve una sola pantalla principal con acceso a todas las siguientes opciones:

1. Perfil
2. Cargar imágenes
3. Comentarios (Timeline)
4. Interacciones
5. Buscar Profile
6. Buscar Hashtag
7. Inbox (mensajería privada)
8. Editar perfil
9. Cerrar sesión

4.5 Perfil de usuario

La sección de perfil debe mostrar de forma clara y completa:

- Foto de perfil
- Nombre completo
- Username
- Edad
- Género
- Fecha de registro
- Cantidad de followers y de following
- Cantidad de publicaciones
- Estado de la cuenta (activa/inactiva)

Programación II

- Botón para Seguir / Dejar de seguir (cuando se visualiza el perfil de otro usuario)

4.6 Cargar imágenes

Las imágenes que sube el usuario se guardan en la carpeta /imagenes de su cuenta. Al subir una imagen, el usuario puede:

- Asignarla a una carpeta personal específica.
- Agregar una descripción (máximo 220 caracteres).
- Incluir hashtags dentro del texto.
- Etiquetar a otros usuarios usando el carácter @ seguido del username.

Cada publicación queda registrada en el archivo insta.ins.

Dimensiones recomendadas (diseño responsive)

El sistema debe adaptar el tamaño de las imágenes según el tipo de vista simulada (variables MODO_MOBILE):

Tipo de imagen	Vista móvil
Cuadrada	1080 x 1080 px
Vertical	1080 x 1350 px
Horizontal	1080 x 566 px
Columnas en el grid de publicaciones	3 columnas

4.7 Comentarios (Timeline)

Esta sección despliega todas las publicaciones ("insta") propias y las de los usuarios que se sigue, ordenadas de la más reciente a la más antigua. El formato de cada publicación es:

USERNAME escribió:

"Contenido de la publicación" — Fecha

Una publicación puede incluir:

- Mención a otro usuario, usando @ seguido del username.
- Un hashtag, marcado con el carácter #.
- Una imagen adjunta.
- Un sticker(CARGAR).

4.8 Interacciones (menciones)

En esta sección se muestran todas las publicaciones de otros usuarios en las que se ha mencionado al usuario actual (usando @seguido_de_su_username), sin publicaciones duplicadas.

4.9 Buscar Profile

Esta sección permite explorar el contenido de otros usuarios del sistema. Por defecto deben existir tres cuentas de ejemplo con contenido temático (por ejemplo, noticias, moda/deporte y entretenimiento), para que la búsqueda tenga resultados desde el primer uso.

a) Buscar personas

Se ingresa un texto y el sistema muestra todos los usuarios cuyo username contenga ese texto (coincidencia parcial). Por ejemplo, buscar "patito" debe encontrar tanto al usuario patito como a elpatitoloco. El listado se muestra con el formato: USERNAME — Lo sigo / No lo sigues.

b) Entrar a un perfil

Al ingresar un username existente se muestran sus datos generales (nombre completo, género, edad, fecha de ingreso), la cantidad de followers y following, y si el usuario actual lo sigue o no. Se ofrecen dos acciones:

- **Seguir:** agrega el username del usuario actual al archivo followers.ins del perfil visitado.
- **Dejar de seguir:** pide confirmación antes de quitar el seguimiento; puede manejarse con un campo booleano dentro de following.ins que indique si el seguimiento sigue activo.

También se puede acceder a la opción Ver sus publicaciones, que muestra en orden de más reciente a más antigua todo el timeline de ese usuario.

4.10 Buscar Hashtag

Se ingresa una palabra y el sistema muestra todas las publicaciones que contienen ese hashtag, con el mismo formato usado en el timeline. El orden cronológico no es relevante en esta búsqueda, pero sí es obligatorio evitar publicaciones duplicadas en el resultado.

4.11 Inbox — mensajería privada

El Inbox es un sistema de mensajería directa similar al "Direct" de Instagram, almacenado en el archivo binario inbox.ins. Cada mensaje guarda:

Campo	Descripción
Username emisor	Quién envía el mensaje
Username receptor	Quién recibe el mensaje
Fecha y hora	Momento del envío
Mensaje	Texto del mensaje (máximo 300 caracteres)
Tipo	Texto o Sticker
Estado	Leído o No leído

El Inbox debe permitir:

- Enviar un mensaje de texto.
- Ver el historial de una conversación.

Programación II

- Marcar mensajes como leídos.
- Eliminar una conversación completa.
- Enviar stickers dentro del chat.
- Mostrar una notificación cuando llegan mensajes nuevos.

4.12 Sistema de stickers

Cada usuario cuenta por defecto con 5 stickers iniciales: Feliz, Triste, Corazón, Risa y Aplauso, guardados en la carpeta `stickers_globales` y registrados en su archivo `stickers.ins`.

Agregar nuevos stickers

- El usuario puede importar una imagen propia desde su computadora.
- El sistema valida que el formato sea `.png` o `.jpg`.
- El sticker se guarda en la carpeta `/stickers_personales` del usuario y se registra en `stickers.ins`.

Enviar un sticker

10. El usuario entra al Inbox.
11. Selecciona una conversación.
12. Elige la opción "Enviar sticker".
13. El sistema muestra la galería de stickers disponibles (por defecto y personales).
14. El usuario selecciona uno.
15. El sticker se guarda en `inbox.ins` con `tipo = STICKER`.
16. Se muestra de forma gráfica dentro del chat.

4.13 Activar / Desactivar cuenta

Desde Editar perfil, el usuario puede cambiar el estado de su cuenta:

- Si la cuenta está activa y se elige desactivar, el sistema pide confirmación antes de aplicar el cambio.
- **Efecto de una cuenta desactivada:** se comporta como si no existiera — no aparece en las búsquedas y ninguna de sus publicaciones o comentarios debe mostrarse en el resto del sistema.
- Si la cuenta ya está desactivada, la opción la reactiva automáticamente (sin pedir confirmación) y muestra un mensaje confirmando el cambio.

4.14 Cerrar sesión

Tras confirmar la acción, se cierra la sesión del usuario actual y el sistema vuelve a mostrar la pantalla de Log In.

5. Requisitos técnicos obligatorios (checklist unificada)

- **Archivos binarios:** usados en la totalidad del almacenamiento persistente del sistema (usuarios, publicaciones, mensajes, stickers, followers/following).

Programación II

- **Hilos:** aplicados como mínimo en el reproductor musical, el organizador de archivos y la revisión de notificaciones del Inbox.
- **Sockets cliente-servidor:** arquitectura propuesta para que el sistema (en especial INSTA+) funcione en red entre varios usuarios, con un servidor central que administra el acceso a los archivos binarios.
- **Listas enlazadas simples:** implementadas y usadas para manejar el timeline, los listados de followers/following y los resultados de búsqueda.
- **Archivos utilitarios:** clases de apoyo para leer y escribir los distintos archivos binarios de forma centralizada.
- **Extensión de archivo propia:** los archivos del sistema deben usar extensiones personalizadas, por ejemplo usuarios.sop.
- **Excepciones propias:** se deben crear al menos 3 excepciones personalizadas (por ejemplo, UsernameDuplicadoException, CuentaDesactivadaException, ArchivoCorruptoException).
- **Diseño visual:** la interfaz debe inspirarse en un sistema operativo real (Windows, Linux o Mac), y la pantalla de INSTA+ debe funcionar como una sola vista integrada, replicando la experiencia de una red social real.